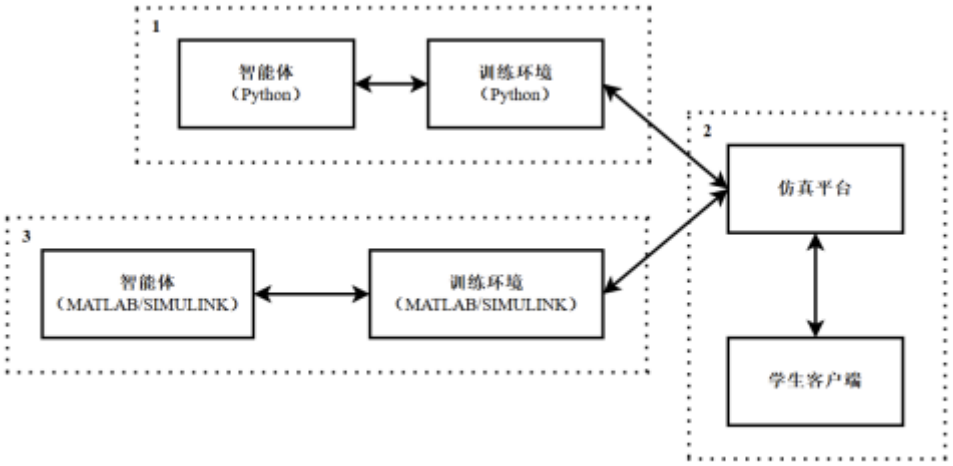
系统设计

本课程《无人系统设计》的大作业基于课程教学实践对战仿真平台，该平台及其外围组件的总体架构和交互结构如下图所示。



其中，虚线框 1 内的部分为基于 Python 的智能空中对抗智能体、智能体训练环境及仿真平台交互子系统（下统称“训练环境”），此部分将挖空提供给选择采用 Python 完成大作业的同学；虚线框 3 内的部分为基于 MATLAB 的训练环境，此部分将挖空提供给选择采用 MATLAB 完成大作业的同学；虚线框 2 内的部分为对战仿真平台及其对战可视化客户端，该部分为训练环境提供对战管理、物理仿真、计算资源调度、对战信息转发等功能。

仿真平台及客户端

学生客户端为基于 Unreal Engine 5 的可视化平台，通过从仿真平台获取的战机位姿数据包，渲染得到实时对战画面。同时，通过在学生客户端上发起建立房间请求，仿真平台将会创建用于与训练环境交互的仿真环境示例。

仿真平台为基于TCP的服务器。在接收到客户端发起的建立房间请求后，仿真平台基于客户端所发送数据包中的maxStepsPerEpisode、maxEpisodes等元数据创建仿真环境实例，并开放端口供训练环境连接到仿真环境。

仿真平台提供三种战机的仿真模型，分别为：Simple（无任何空气动力学与物理仿真）、Junior（加入部分战机空气动力学仿真）以及 Jsbsim（包含完整的战机空气动力学模型以及物理仿真）。

仿真环境中有两架仿真战机，分别是由训练环境中智能体控制的战机与靶机，靶机的行动模式与通过学生客户端建立的房间种类相关。

在仿真环境中将会不断进行对战，一次对战对应训练环境中的一个 Episode，对战的一个回合对应训练环境中的一个 Step。仿真环境的对战次数与回合数上限由在学生客户端上创建时的设置决定。在一次对战中，双方战机各自拥有1000点血量，当靶机血量降至0或者达到回合数上限（也即maxStepsPerEpisode），则本次对战结束。当对战次数（也即maxEpisodes）达到上限，则仿真环境将被回收，本次训练结束。

仿真环境与训练环境之间的网络协议见下文。

MATLAB

基于 MATLAB 的训练环境使用TD3算法构建Agent。在训练开始时， MATLAB Simulink 模型 rlflight 将向仿真平台发送 initial 数据包，告知仿真平台两台战机的初始位姿和速度。随后 rlflight 从仿真平台获取战场数据包，从而得到原始观察数据，经过处理后，获取传输给智能体的观测量。智能体获取观测量后，输出控制量给仿真平台，仿真平台由此得到下一组观测数据，从而进行训练。训练过程需要仿真平台和训练环境完成本时间片的更新任务后才能继续。

训练环境与仿真平台之间的通信协议如下：

初始包（训练环境->仿真平台）

```
struct InitData{
    int32_t m_room_uid;                                          //房间id
    int32_t m_unit_id;                                           //战机id
    int32_t m_unit_1[12];                                        //训练战机初始位置与速度信息
    int32_t m_unit_2[12];                                        //靶机初始位置与速度信息
    uint8_t m_padding[(100-2-12-12) * sizeof(int32_t)];          //预留的空间
}
```

对应 MATLAB 中 get_my_initial 函数，在 rlflight 中的send initial pack 中被调用。

战场数据包（仿真平台->训练环境）

```
struct BattleData{
    double m_unit_1[13]{};  //训练战机此时位置与速度信息
    double m_unit_2[13]{};  //靶机此时位置与速度信息
    double m_is_done{};     //对战终止flag
}
```

对应 rlflight 中 TCP Receive 模块的输出与 get_my_observations 等模块的输入。具体参数与物理量的对应关系如下：

```
     m_unit(1-3)    //x\y\z坐标
     m_unit(4-6)    //欧拉角φ\θ\ψ对应翻滚(roll)、俯仰(pitch)、偏航(yaw)
     m_unit(7-9)    //线速度u\v\w
     m_unit(10-12)  //角速度ω\β\η
     m_unit(13)     //血量
```

战机控制包（训练环境->仿真平台）

```
struct CtrlData{
    double m_throttle;  //油门，[0,1]
    double m_pitch;     //俯仰，[-1,1]
    double m_roll;      //滚转，[-1,1]
    double m_yaw;       //偏航，[-1,1]
    double m_is_done;   //对战终止flag
}
```

对应 rlflight 中 RL Agent 与 get_my_isdone 的输出。

训练流程（以 MATLAB 为例）

1. 运行学生客户端，声明回合数上限与对战次数上限。





2. 创建仿真环境，记录仿真环境对应端口。



3. 在 MATLAB 中，运行 env.m。这一步将会创建用于训练环境的 TD3 Agent 以及用于强化学习训练的 Simulink 模型。
4. 修改 set_ip.m 中port参数为仿真环境对应端口并运行。

```
% 设置TCP模块的IP和端口号
% ip为战斗服务器IP，本机训练为'127.0.0.1'
% port为战斗服务器端口号，默认为1000
ip = '10.119.13.38';
port = '1000';
set_param('rlflight/Send Initial Pack/TCP Send Initial Pack','Host',ip);
set_param('rlflight/Send Initial Pack/TCP Send Initial Pack','Port',port);
set_param('rlflight/Training/TCP Send Control Pack','Host',ip);
set_param('rlflight/Training/TCP Send Control Pack','Port',port);
set_param('rlflight/Training/TCP Receive Observation Pack','Host',ip);
set_param('rlflight/Training/TCP Receive Observation Pack','Port',port);
```

5. 运行 training.m ， 开始训练。

MATLAB 具体的参数设置以及代码说明可参考文件中的注释。

MATLAB文件结构及说明

在开始训练之前，请在matlab左侧文件树中右键项目文件夹-添加到路径-选定的文件夹与子文件夹

```
--utils
 --createNetworks.m  //创建TD3Agent的工具函数，用于创建Actor与critic网络
 --createTD3Agent.m  //创建TD3Agent，主要负责设置各个参数
 --normalization.m   //用于normalize_my_obs的工具函数
 --not_zero.m        //用于判断double类型是否为0
 --saveCommand.m     //用于保存工作区的Agent
--training
 --envs.m            //创建用于强化学习训练的Agent与simulink模型
 --rlflight.slx      //simulink模型，负责matlab训练环境中各个组件的交互
 --set_ip.m          //修改训练环境与仿真平台连接参数
 --training.m        //开始训练
--get_my_reward.m    //以下均为TODO
--get_my_isdone.m
```

```
--get_my_obs.m
--get_my_initial.m
--normalize_my_obs.m
```

Python文件结构及说明

transformer_option_agent.py:

- The agent is trained using the transformer option agent.
- Support SAC, PPO, TD3 algorithms.

algs.yaml:

- configure algorithm parameters

envs.yaml:

- configure environment parameters

custom_model.py:

- define custom model

callback.py:

- define callback functions to save the reward
- More information can be found on the use of stable baselines3

visualizer.py:

- visualize the flight path

Dependencies:

- anaconda
- python 3.9
- pytorch 2.2.1 with cuda 12.1: conda install pytorch2.2.1 torchvision0.17.1 torchaudio==2.2.1 pytorch-cuda=12.1 -c pytorch -c nvidia
- stable baselines 3: pip install stable-baselines3[extra]
- gymnasium: pip install gymnasium